UV-Decomposition Recommendation System

Data Mining Final Project

Angel Coronel 24614703

May 23, 2025

Abstract

We can see useful patterns using recommendation algorithms in large rating matrices. In section 9.4 of *Mining of Massive Datasets*, we see how this can be approached using UV decomposition where a user matrix U and item matrix V can be used to find product approximations. This algorithm is used in this project to apply stochastic gradient descent for matrix factorization while minimizing the root mean square error (RMSE) of the reconstruction with L_2 regularization to mitigate overfitting.

Here, the MovieLens 100K dataset is used to explore and implement this algorithm. The dataset has 100,000 rating from 943 users on 1,682 movies. An RMSE of 0.93 is achieved with five training epochs with 32 latent factors, a learning rate of 0.01, and a regularization weight $\lambda = 0.05$. Furthermore, the model's top 10 recommendation performance outperformed the item popularity heuristic by 25%. These results show that UV decomposition is a very useful tool for modern recommendation systems.

1 Problem to Be Solved

1.1 Context

Users are constantly exposed to catalogs with hundreds of seemingly random things. Just because online platforms can show any user any random thing does not mean it is good for either the user or the platform's business. That's why both the platform and users depend on automated recommendation systems to show few items that the user is more likely to care about. The utility matrix $M \in \mathbb{R}^{n \times m}$ is the data structure whose input m_{ij} keeps track of user i's exact rating or preference for item j. Most of the input is blank and real world matrices are commonly 99% sparce.

Hence, the recommendation system has to infer whatever is missing (rating-prediction view) by identifying a subset of items with the highest expected rating for each user (top-k ranking view). Platforms get more business value when they have higher accurate predictions like Netflix's famous offer of $1 million for a 10% RMSE bump in their own algorithm.

1.2 Formal Definition

Let:

- $U = \{u_1, \ldots, u_n\}$ be the set of users,
- $I = \{i_1, \ldots, i_m\}$ be the set of items, and
- $R \subseteq U \times I$ the set of observed (u, i) pairs.

Given a partially observed rating function

$$r : R \longrightarrow \mathbb{R},$$

the goal is to learn a predictor

$$\hat{r} : U \times I \longrightarrow \mathbb{R}$$

that minimizes a loss such as root mean square error (RMSE):

$$\text{RMSE}(\hat{r}) = \sqrt{\frac{1}{|R_{\text{test}}|} \sum_{(u,i) \in R_{\text{test}}} (r(u,i) - \hat{r}(u,i))^2}.$$

In english terms, we look for $\hat{M}$ that gets close to M in what can be seen and generalize what can't be seen. UV decomposition constrains $\hat{M} = UV$ with low rank $d \ll \min(n, m)$, making the goal be learning the matrices $U \in \mathbb{R}^{n \times d}$ and $V \in \mathbb{R}^{d \times m}$.

2 Algorithm — UV Decomposition

2.1 Assumptions & Model Equation

The textbook shows that the $n \times m$ utility matrix M can be approximated by the product of two "long-thin" matrices

$$M \approx \hat{M} = UV, \qquad U \in \mathbb{R}^{n \times d},\ V \in \mathbb{R}^{d \times m},\ d \ll \min(n, m).$$

Here each row of U mixes a user's weights on d factors while each column of V mixes how much an item indicates to those factors. The info needed from $\mathcal{O}(nm)$ is reduced to $\mathcal{O}(d(n+m))$ parameters. With the smaller amount of parameters to consider, the data is less sparce.

2.2 Bias & Regularization

To get U and V, the regularized squared error is minimized over the observed ratings with the following:

$$\mathcal{L} = \sum_{(u,i)\in R} (r_{ui} - \mu - b_u - b_i - \langle U_u, V_i \rangle)^2 + \lambda \left(\sum_u \|U_u\|_2^2 + \sum_i \|V_i\|_2^2 + \sum_u b_u^2 + \sum_i b_i^2 \right),$$

where:

- r_{ui} is the known rating for user u and item i,
- μ is the global average rating,
- b_u and b_i are user and item bias terms,
- $\langle \cdot, \cdot \rangle$ is the dot product,
- $\lambda > 0$ is the L_2 regularization used to reduce the amount of overfitting.

After getting rid of biases, this simplifies everything into minimizing RMSE.

2.3 Optimization Strategy

Stochastic Gradient Descent (SGD) is used to optimize the parameters. The Colab implementation follows Netflix's update loop. For each observed triple (u, i, r_{ui}):

1. Compute the prediction: $\hat{r}_{ui} = \mu + b_u + b_i + \langle U_u, V_i \rangle$,
2. Compute the error: $e = r_{ui} - \hat{r}_{ui}$,
3. Then update the parameters:

$$\begin{aligned} b_u &\leftarrow b_u + \eta(e - \lambda b_u), \\ b_i &\leftarrow b_i + \eta(e - \lambda b_i), \\ U_u &\leftarrow U_u + \eta(eV_i - \lambda U_u), \\ V_i &\leftarrow V_i + \eta(eU_u - \lambda V_i), \end{aligned}$$

where η is the learning rate. This pseudocode is a form of the element-wise optimum from the textbook and use only the non-blank entries with a time complexity of $\mathcal{O}(d\,|R|)$.

3 Performance

3.1 Why UV Factorization Works

Since users only care about a limited amount of things like genre, brand or price then the big utility matrix can be embedded in a much smaller space. The UV decomposition does the same thing. In UV decomposition, each user's preferences are a weight vector U_u, each item's characteristics are a feature vector V_i and their dot product gives an estimation of the rating:

$$\hat{r}_{ui} = \langle U_u, V_i \rangle.$$

. Through we experimentation we can note that as soon as $d \approx 20-100$ the reconstruction error clearly falls. This shows that a small basis captures the dominant variation in ratings.

3.2 Time & Space Complexity

Aspect	**SGD Variant (Colab)**	**ALS Variant (Batch least squares)**
Time per epoch	$\mathcal{O}(d\,\|R\|)$	R
Memory	$\mathcal{O}(d(n+m)) + \mathcal{O}(\|R\|)$	R
Parallelism	Fine grained (shuffle ratings)	Coarse grained (solve users then items)

Table 1: Comparison of SGD and ALS Variants

The optimization done in SGD for a single element shows the update function where only the corresponding ratings in rows or columns are look at. This ensures optimization to be local and inexpensive. On the other hand, storage is linear consistently with the amount of users & items. This linear growth is good enough for millions of each to fit in memory on modern servers.

3.3 Strengths & Weaknesses

To be brief, UV decomposition has an efficient balance between what its capable of doing and its limits. It is mathematically straightforward, very

useful, computationally efficient and could be further enhanced.

Strengths	**Weaknesses**
Grows linearly with observed ratings	Cold start struggles with new users/items without some starting info
Produces dense vectors that power downstream tasks	Quality depends on good hyperparameter search
Handles high sparsity by sharing information globally	Temporal drift is ignored unless factors are retrained or time awareness is introduced
Simple dot-product form allows fast scoring and A/B testing	Popularity bias and unbalanced data can skew factors

Table 2: Strengths and Weaknesses of UV Decomposition

4 MovieLens 100K Case Study

4.1 Dataset Description & Preprocessing

The MovieLens 100K contains 100,000 1 to 5 star ratings recorded. The body is made up of 943 users on 1,682 movies with 19 genre flags. The data is sliced into 80%/20% parts for training and testing.

The textbook was used to figure out how to remove bias for pre-processing:

1. Subtract each $r_{uj} - \mu_u$ to apply user mean-centering to get rid of grade-inflation effects.

2. Remove bias of item-mean on what is left to address any popular movies' bias

3. Bring back the two offsets after prediction to obtain calculated outputs.

4.2 Experiment Setup and Results

Here's the set up for the project:

Component	Setting	Reasoning
Latent dimensionality d	$\{8, 16, 32, 64\}$	Aim for capacity vs. over-fit
Optimizer	SGD	Lightweight vs full ALS
Learning rate	0.007 with $\eta \rightarrow 0.5\eta$ when RMSE plateaued 3 epochs	Textbook's "half-step"
Regularization λ	$\{0.02, 0.05, 0.1\}$ L_2 on U,V	L_2 control overfitting
Early stop	RMSE < 0.0005 for 5 epochs	follows convergence rule §9.4.5
Evaluation	RMSE / MAE (prediction) and Precision-K, Recall-K (top-N)	Measures both prediction and ranking quality

Table 3: Experiment Settings

Some filtering baselines like global mean, user mean, item mean, and cosine item-item collaborative filtering (CF) ($k = 40$) were redone for context.

Here are the results from the project.

Model	**RMSE**	**MAE**	RMSE vs. UV	**Notes**
Global mean	1.03	0.82	+0.10	naive baseline
User mean	1.01	0.80	+0.08	captures rater bias
Item mean	1.00	0.79	+0.07	popularity only
Item-Item CF ($k = 40$)	0.98	0.77	+0.05	textbook memory-based CF
UV SGD ($d = 32$, $\lambda = 0.05$)	**0.93**	**0.74**	—	best model

Table 4: Model Evaluation Results (95% CI for RMSE: ± 0.006)

K	**Precision-K**	**Recall-K**
5	0.34	0.16
10	0.28	0.21
20	0.22	0.30

Table 5: Top-N Recommendation Quality (Compared to Item-Item CF: P-10 = 0.19, R-10 = 0.15)

4.3 Discussion

Overall, the model did very well since UV decomposition reduces about 5% RMSE compared to memory-based CF and about 10% compared to naive baselines. This is similar to the ˜7% bump for Netflix contestant using the same idea.

UV decomposition works because the 32 factor model does a good job at capturing matrix dimensions with only about 84k parameters ($(943+1682)\times 32$) instead of 100k observations. Which in turn, makes the algorithm be very scalable with each SGD epoch touching —E— samples to $\mathcal{O}(|E|d)$ FLOPs and training finished in 40 epochs (took about 30 seconds each time on Colab). Space is linear in $(n+m)d$.

The top-N utility has the latent factors improve hit-rate at all K. The utility really gets the precision well which makes for better ranking of things that are actually relevant to a user or a business.

However, the performance doesn't do well for cold starts with users who have less than 10 ratings. This is because user vectors don't have enough information to make good estimations. This may be a good reason why some platforms will ask for a minimum amount of preferences given by the users as soon as they create an account.

5 Conclusion

The coding experiment uses a public benchmark to show that UV decomposition can be used an effective modern recommendation system. This paper shows how the problem is framed, a description of the algorithm, a cost analysis and an example using this algorithm on a large dataset.

References

[1] J. Leskovec, A. Rajaraman, and J. D. Ullman, *Mining of Massive Datasets*, 3rd ed. Cambridge University Press, 2020. Chapter 9.

[2] S. Zhang, L. Yao, A. Sun, and Y. Tay, "Deep Learning based Recommender System: A Survey and New Perspectives," *ACM Computing Surveys*, vol. 1, no. 1, pp. 1–35, Jul. 2018.

[3] Y. Koren, R. Bell, and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," *IEEE Computer*, vol. 42, no. 8, pp. 30–37, Aug. 2009.

[4] N. Hug, "Surprise: A Python scikit for recommender systems," [Online]. Available: https://surpriselib.com/

[5] Lyst, "LightFM: A Python implementation of popular recommender algorithms," [Online]. Available: https://making.lyst.com/lightfm/

[6] Scikit-optimize contributors, "scikit-optimize: Sequential model-based optimization in Python," [Online]. Available: https://scikit-optimize.github.io/

A Hyper-parameter Grid

The best model was achieved with $d = 32$, $\eta = 0.007$, and $\lambda = 0.05$. In order to avoid overfitting, early stopping and a learning rate halving were applied.

Latent Factors (d)	**Learning Rate** (η)	**Regularization** (λ)	**RMSE**	**MAE**
8	0.007	0.02	0.98	0.79
16	0.007	0.02	0.96	0.77
32	0.007	0.05	**0.93**	**0.74**
64	0.007	0.05	0.94	0.75
32	0.003	0.05	0.95	0.76
32	0.007	0.10	0.94	0.75

Table 6: Grid of Searched Hyperparameters and Performance Metrics

B Source Code

Colab Notebook Link for UV Decomposition Project